BSc (Hons) in Information Technology

Year 1

Data Structures and Algorithms – IT1170

Practical 3 – Stacks

Objectives:

- Understand the stack data structure and its operations.
- Implement stack operations using arrays in Java.
- Apply stack concepts to solve real-world problems.
- Develop problem-solving skills through hands-on coding exercises.

Stacks

A **stack** is a linear data structure that follows the **Last In, First Out (LIFO)** principle. This means that the last element added to the stack is the first one to be removed.

Basic Stack Operations:

i. **Push** – Adds an element to the top of the stack.

ii. **Pop** – Removes and returns the top element of the stack.

iii. **Peek (Top)** – Returns the top element without removing it.

iv. **isEmpty** – Checks if the stack is empty.

v. **isFull** – Checks if the stack is full (for array implementation).

Stacks are commonly used in applications such as function calls, expression evaluation, and undo mechanisms in software.

Exercise 1: Basic Implementation of Stack

1. Create a class StackArray that implements a stack using an array and stores integers.
2. Implement the following methods:
    - push(int item): Adds an item to the stack.
    - pop(): Removes and returns the top item.
    - peek(): Returns the top item without removing it.
    - isEmpty(): Checks if the stack is empty.
    - isFull(): Checks if the stack is full.
3. Introduce a new method named display() to print the elements in the stack.
4. Write a main method to test the stack operations.

Sample output:

```
Stack elements: 10 20 30
Peek: 30
Popped: 30
Stack elements: 10 20
Is stack empty? false
Is stack full? false


...Program finished with exit code 0
Press ENTER to exit console.
```

Exercise 2: Reversing a String

5. Modify the above program to take a string input. Your program needs to take the input from the user and print the reversed user input. *Hint: You may have to change the stack to store characters.*

Sample Output:

```
Enter a string: hello world
Reversed string: dlrow olleh

...Program finished with exit code 0
Press ENTER to exit console.
```

Exercise 3: Checking balanced parentheses

6. Assume the above StackArray class. You need to introduce a new method to this class to check whether parentheses are balanced. Write Java code segment for ***boolean isBalanced(String str)*** to take a string input from the user. Using the stack data structure your program should return true for balanced parentheses i.e. {[()]}, and false for imbalanced parentheses i.e. {{(()])].

Sample output:

```
Enter an expression with parentheses:
{{(]]}
The expression has imbalanced parentheses.


...Program finished with exit code 0
Press ENTER to exit console.
```